

NHF-Tomb of the Mask

Felhasználói dokumentáció

A program a Tomb of the Mask nevű játék hasonmása. Ha elindítjuk a programot, akkor a főmenü fog fogadni minket, ahol láthatjuk a játék nevét, egy Play gombot és egy Quit gombot. Ha a Quit gombra klikkelünk, akkor a játék befejeződik és bezáródik az ablak. Ha viszont a Play gombra kattintunk, akkor átkerülünk az almenübe, ahol a szinteket találhatjuk.

Az almenü jobb felső sarkában megfigyelhetünk egy vissza gombot, amely azért felelős, hogy vissza tudjunk lépni a főmenübe. A szintek középtájt, sárga kockákban sorban megszámozva helyezkednek el. Ha ráklikkelünk valamelyik szintre, akkor az annak megfelelő pályát fogjuk látni.

Tehát a megfelelő szint kiválasztása után a játék nézetbe kerülünk. Itt fog lefutni a program lényegi része a játékmenet. Itt is a jobb felső sarokban láthatunk egy Pause gombot, amely arra szolgál, hogy ki tudjunk lépni a főmenübe, ha esetleg nem azon a pályán szeretnénk játszani, vagy csak szimplán be szeretnénk fejezni a játékot. A játék nézet felső részén láthatjuk, hogy melyik pályán vagyunk. Például, ha a 3. szintet indítottuk el, akkor a képernyőn egy Level 3 feliratot láthatunk sárga színnel kiírva. Alatta láthatjuk teljes egészében a szinthez tartozó pályát. A pálya lila falakból, egy szürke célból és egy citromsárga karakterből áll. A lényeg az, hogy a karakterünket eljuttassuk a célba. A karaktert a W A S D gombokkal tudjuk irányítani. A W felfelé, az S lefelé, az A balra, a D pedig jobbra fogja mozgatni a karakterünket. A karakter nem szimplán csak arrébb megy egy kicsit, ha lenyomjuk valamelyik gombot, hanem egészen a lenyomott irányban található első falig fog csúszni és a fal előtt meg fog állni, ahonnan várja a következő utasítást. Ha sikerült végig csúszkálnunk a pályán és beértünk a célba, akkor kikerülünk az almenübe, ahonnan elindítottuk a pályát ezzel jelezvén, hogy sikeresen teljesült a szint. Ezek után indíthatunk egy új szintet, vagy kiléphetünk a játékból.

Ha a kilépés mellett döntöttünk, akkor találkozhatunk a fájlok között egy statisztika nevezetű szöveg fájlal, amiben az előbb leállított játékmenet statisztikái láthatók. Pontosabban az összesen teljesített szintek száma és a játékban eltöltött idő. A játékidő a játékban eltöltött pillanatok nagyságától függ. Tehát, ha csak pár másodpercet töltöttünk el benne, akkor másodpercet fog kiírni, ha kicsit többet akkor már percet, ha pedig nagyon sok időt, akkor már órában láthatjuk ezt a számot.

Programozói dokumentáció

- A dokumentációban részletesen láthatjuk a Tomb of the Mask nevű C program felépítését, amely SDL2 könyvtár használatával jeleníti meg a játékot.
- A program 6 különböző modulra van bontva, melyeket a **main.c**-ben fűzünk össze. A szükséges modulok: **ablak.c/h**, **alakzatok.c/h**, **beolvas.c/h**, **iranyitas.c/h**, **menu.c/h** és a **statisztika.c/h**.
- A mainben először is az ablak megjelenítésére kerül sor. A renderer és az ablak pointer létrehozása után, az ablak.c-ben található ablak nevű függvényt hívjuk meg. A függvénynek szüksége lesz egy felíratra, ami az ablak tetején jelenik meg, a méretekre és a 2 pointerre, amelyet az előbb létrehoztunk. A függvény ezután létrehozza a megadott ablakot és azt is megvizsgálja, hogy minden rendben történik-e a létrehozás közben. Ha nem, akkor hiba üzenetet fog adni.
Ezek után szükségünk lesz egy időzítőre, ami később majd arra szolgál, hogy usereventeket generálva elakadás nélkül faltól falig csússzon a karakter. Ezt a függvényt is az ablak.c-ben hozzuk létre.
A következő lépés egy mátrix struktúra létrehozása, amely a beolvasás headerjében található paraméterek alapján hozza létre azt.

Struktúrák és enumok

- A mátrix struktúra egy pointerre mutató int pointert, egy oszlop és sorszámot tároló intet és egy karakter struktúrát tartalmaz. Az utóbbit azért tartalmazza, hogy könnyebben tároljuk és leptsük a karakterünket a mátrixban.
- A karakter struktúrát szintén ebben a modulban hozzuk létre, amely x és y koordinátáit tartalmazza a karakternek és azt, hogy milyen irányba menjen.
- Ezt egy enumként hozzuk létre szóval szükségünk lesz egy irány enumra, amiben a fel, le, jobbra, balra és áll utasítások állnak. Ennek a segítségével egyszerűen eldönthetjük, hogy melyik irányba mozog, vagy áll a karakter.
- További struktúrákra lesz még szükségünk. Ebből az egyiket az alakzatok.h-ban hozzuk létre, mely a gombok adatait fogja tárolni. Ez alatt x, y koordinátára, méretére, esetlegesen egy szövegre, amely rajta áll és egy lenyomást követő utasításra lesz szükségünk. Ezzel a struktúrával könnyedén hozzuk majd létre a gombjainkat.
- Szükségünk lesz még egy statisztikák nevű struktúrára, amelyben teljesített szintek számát, a kezdeti időt, a bezárási időt és ezeknek a különbségét tároljuk.
- Ezen kívül még 2 enumra lesz szükségünk, egy menü nevezetűre, amelyben a menüket tároljuk, és egy pályákra, amelyben pedig a pályákat tároljuk.

Ha ezekkel megvoltunk akkor a main-ben be kell olvasnunk a mátrixunkat a szövegfájlból.

Mátrix beolvasása

- A mátrixot dinamikusan foglaljuk le a memóriában. Erre a **beleolvasas** nevű függvény felel. Ennek a függvénynek át kell adnunk a szövegfájlt, amelyben a pályát írjuk le. A pálya 1-esekből, 0-ákból, 3-asból, és 2-esből áll. Az 1 a fal, a 0 a szabad terület, a 2 a cél a 3 pedig a karakter kezdőhelye. A szövegfájlt soronként beolvassuk, ezzel megkapva a sorok számát, emellett számoljuk azt is, hogy hány karakterből áll és a végén ezt elosztjuk a sorok számával, amivel megkapjuk, hogy hány oszlopa lesz a mátrixnak. Utána lefoglaljuk a mátrix helyét a memóriában.
- A következő függvénnyel pedig fel fogjuk tölteni a mátrixunkat. Ez lesz a **matrix_feltolt**. Ekkor bejárjuk a szövegfájlt és beleolvassuk a lefoglalt memóriába a számokat. Ha 3-assal találkozunk, akkor azt átírjuk 0-ra és a karakternek pedig megjegyezzük ezt a pozíciót, hiszen innen kell kezdenie.
- A mátrixot tesztelés kedvéért kirajzolhatjuk a konzolra, ezért a **matrix_kirajzol** felel, de ennél fontosabb a grafikus megjelenítése. Ehhez a **palya_kirajzol** függvényt használjuk. Ez bekér egy renderert, a mátrixot, és a main-ben létrehozott méretet, amely arra szolgál, hogy mekkora legyen egy szám a mátrixból a képernyőn. Először bepozícionáljuk középre a pályát, ezután pedig bejárjuk a mátrixot. A bejárás előtt még létrehozunk egy blokkot, amely segítségével részletesebben ki tudjuk rajzolni a pályát. Ezt az **SDL_Rect egyBlock** teszi meg. Ezek után minden szám helyén a mátrixban a megfelelő színekkel lerendererjük a blokkot így kialakul a teljes pályánk. Fontos lesz a program végén felszabadítani a mátrixot így egy kis könnyebbségért ezt is függvénybe foglaljuk, ami a **matrix_felszabadit** névre hallgat.

Lefutás

- A játék lefutására egy while ciklust használunk. Létrehozzuk a **fut** változót és ha kilépünk akkor azt false-ra állítjuk, melynek hatására leáll a program. Szükségünk lesz egy eventre és switch-casekre. Először megvizsgáljuk, hogy ha lenyomjuk az egér bal gombját akkor mi történik.
- Erre létrehozzuk a **gomb_vizsgalat** függvényt, amely egy **rajtavan** függvény segítségével (ez az egér és a gombok x, y koordináit hasonlítja össze) meghatározza, hogy lenyomtuk-e a gombot. Minden előre meghatározott gombra ezt megvizsgáljuk. A függvénynek szüksége lesz az eventre, a mátrixra, egy jelenlegi menüre, a jelenlegi pályára és a futra.
- Tehát még ez előtt létrehozzuk a **menuk_jelenlegi_menu**, hogy midnig a megfelelő menübe kerüljünk, ha olyan gombra nyom a felhasználó. Mivel enum-ként hoztuk létre ezt, így könnyedén lépkedünk a menük között, mivel a főmenü értéke 0, az almenüé 1, a játék nézeté pedig 2. Ezek után a **palyak_jelenlegi_palya** is létrehozzuk, hogy hasonló képpen használjuk fel, mint a menüt.
- A ciklusban ezután ki kell renderelnünk az egyes menüket. Ezt egy újabb switch-case-el hajtjuk végre. A switch a jelenlegi_menuket fogja figyelni. Mindegyik menü kinézetét az alakzatok modulban meghatározzuk. Ezen függvények lesznek a **fomenu**, **almenu**, **jatekmenu**.
- Ha a jelenlegi_menu az 2-es értéket vesz fel akkor külön meg kell csinálnunk a játék irányítását.

Irányítás

- A **jatek_iranyitasa** nevezetű függvény bekéri az eventet, a mátrixot, a menüt és a statisztikát is. A függvényben switch-case-el megvizsgáljuk, hogy melyik billentyűt nyomjuk le, és arra mit kell reagálnia a karakternek. Itt beállítjuk a mátrixban a játékos irányát a lenyomott irányra.
- Mivel a karakter faltól falig mozog, ezért vizsgálnunk kell azt, hogy meddig csúszik a karakter a 0-ákon, mert ha a következő elem a mátrixból 1-es lesz, akkor az az előtti 0-án meg kell állnia. Az irányoknak megfelelően vonogatunk, vagy hozzáadunk a mátrix x, y koordinátaíhoz, ezzel elérve, hogy a karakter pozíciót váltson.
- Ha ez már tökéletesen működik akkor ki kell térnünk arra is, hogy ha a mátrix következő eleme 2-es lesz akkor a karakter bejutott a célba. Itt egyszerűen összevetjük a karakterünk jelenlegi helyét a mátrixban található elemmel és, ha az 2-es akkor, a menüt 1 re állítjuk, hogy kikerüljünk az almenübe. Emellett itt növeljük a statisztikánk teljesített szintek értékét is.

Statisztika

- Itt a struktúráként létrehozott **statisztikak** paramétereit fogjuk változtatni. A **statisztika_kiiras** függvény várja a statisztika struktúrát és egy fájl nevet, amelybe írni fogja a statisztikákat.
- A függvény használ time.h beli függvényeket ezért azt includeolnunk kell. A játszott időt úgy állapítja meg, hogy a program elindításakor a statisztika indítás paramétert egyenlővé teszi az akkori idővel. Ha pedig bezárjuk a programot akkor a bezárás paramétert pedig azzal az időponttal teszi egyenlővé. Ezt a **time()** függvény teszi meg. Ezek után pedig a **mennyiJatszott** paramétert egyenlővé tesszük a két időpont különbségével és kezdőthet a fájlba írás.
- A fájl kap egy keretet, amelyen belül kiíratjuk a játék időt és a teljesített szintek sorszámát. Ez a keret nem fog elcsúszni, ha mondjuk 1 jegyű vagy 2 jegyű számok kerülnek a fájlba. Tehát itt külön vizsgálnunk kell ezeket az eseteket, hogy a fájl mindig szépen esztétikusan nézzen ki.

Bezárás

- A program legvégén már csak arra kell ügyelnünk, hogy mindent zárjunk be vagy szabadítsunk fel a megfelelő módon. Ehhez létrehozzuk a **bezarasok** függvényt, amely azért ügyel, hogy felszabadítsa a mátrixunkat, bezárja az ablakot és a betűstílusokat. Itt még ügyeljünk arra, hogy az időzítő bezárására is szükség van.

Szövegek

- A játékban találkozhatunk szövegekkel is melynek kiírásáért a **text** nevű függvény felel. A függvény bekéri a renderert, egy betűstílust, melyeket a mainben a **TTF_Init()** függvény alatt létre kell hoznunk, ezen felül bekéri a megjelenítendő szöveget, a szöveg színét és a koordinátáit.
- A színeket előre definiálhatjuk az **SDL_Color szín** segítségével.